

# FAQ-23: How Do I Monitor Box With Dynatrace?

**Series:** FAQ — Frequently Asked Questions | **Reference:** 23 — Monitoring Box With Dynatrace | **Created:** August 2026 | **Last Updated:** 08/27/2026

## Overview

Box holds the documents. That is the whole reason this integration comes up: the audit trail of who opened, downloaded, shared, and deleted them is a compliance artifact, and it lives in a system nobody in the observability team administers. The request usually arrives phrased as a logging problem — "we need Box logs in Dynatrace" — and is answered as though the only open question were which ingestion endpoint to point at.

It is not. Two things about this integration are commonly assumed the wrong way round, and both change the design:

- **There is no turnkey Dynatrace integration for Box.** No Hub listing, no extension to activate. Verified three independent ways in August 2026 — Hub catalog search, web search, and the Dynatrace documentation assistant — all empty. You are building and owning this.
- **Box cannot push its audit feed to you.** Box webhooks exist, and they are the wrong tool: they attach to individual files and folders, cap at 1,000 resources per application, cannot be set on the root folder, and never carry logins, admin actions, or security alerts. The enterprise audit feed is available *only* by polling, and Box states plainly that it "does not support long polling." Every working design is a scheduled pull that owns a cursor.

That second point is the expensive one. A team that starts from "set up a webhook" will build a receiver, wire it correctly, and then discover that the events they actually needed were never eligible to arrive. This entry exists partly to prevent that specific week.

This entry is a **worked example of FAQ-19**. The generic decisions — signal classes, route selection, extract-then-discard, bucket isolation — are made there; here they are

made concretely for Box, with a sizing example at **1,000,000 events per day** drawn from a real migration question.

**A note on the DQL in this entry.** Box data will not exist in your tenant until the collector runs. Every query below is therefore written to run standalone against a synthetic record using `data record(...)` — **zero bytes scanned, zero cost** — so you can validate the shape before you have ingested anything. All queries were executed against a live tenant on 08/25/2026. Swap `data record(...)` for `fetch logs | filter log.source == "box.events"` once data flows.

---

## Table of Contents

---

1. [Short Answer](#)
  2. [What Box Is — and Why There Is No Turnkey Integration](#)
  3. [The Four Signal Classes](#)
  4. [The Events API — The Constraint That Shapes Everything](#)
  5. [Sizing: A Worked Example at 1,000,000 Events/Day](#)
  6. [Choosing the Collector — Workflow or Python](#)
  7. [Implementation A — Dynatrace Workflow](#)
  8. [Implementation B — Python](#)
  9. [Ingestion, Processing, and Storage](#)
  10. [Field Mapping Starting Point](#)
  11. [Box Shield Alerts](#)
  12. [Querying Box Events in Grail](#)
  13. [Operating the Collector](#)
  14. [Availability Monitoring](#)
  15. [Dashboards, Alerting, and SLOs](#)
  16. [Implementation Sequence and Checklist](#)
  17. [Common Gotchas](#)
  18. [Recommended Approach](#)
-

# Prerequisites

| Requirement           | Details                                                                                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Box licensing         | Business plan or above for the Events API. <b>Box Shield is a separate SKU</b> — § 11 explains what its absence removes, and it is worth confirming before scoping                                                                      |
| Box admin access      | The service account must be an <b>enterprise admin or co-admin</b> holding <i>"Run new reports and access existing reports."</i> Without it the API returns nothing useful — which looks identical to "no events" (§ 17)                |
| Box application       | A Box app configured for <b>Client Credentials Grant</b> (server-side auth, no user interaction), authorized in the Box Admin Console                                                                                                   |
| Dynatrace environment | SaaS with <b>Grail</b> and <b>OpenPipeline</b>                                                                                                                                                                                          |
| Outbound connectivity | If collecting via Workflow, <code>api.box.com</code> must be on the JS-runtime outbound allowlist (§ 7) — a silent-failure trap                                                                                                         |
| Security sign-off     | This feed carries user identity, file names, and IP addresses. Decide retention and masking before building — see FAQ-19 § 4 and § 6                                                                                                    |
| Related reading       | <b>FAQ-19</b> (the generic pattern this instantiates), <b>FAQ-20</b> (the Zscaler worked example), <b>OPIPE / OPLOGS</b> (OpenPipeline depth), <b>ORGNZ</b> (buckets), <b>WFLOW</b> (workflows), <b>ALERT / SLO / DASH</b> (downstream) |

## 1. Short Answer

Build a scheduled collector that polls the Box Events API and posts to the Dynatrace Log Ingestion API. At typical enterprise volume — including **1,000,000 events per day** — a Dynatrace Workflow is sufficient and needs no external infrastructure.

The five decisions, made:

| Decision  | Answer                                              | Why                                                                          |
|-----------|-----------------------------------------------------|------------------------------------------------------------------------------|
| Transport | Scheduled <b>pull</b> from <code>GET /events</code> | Box has no push for enterprise events. Webhooks cannot carry this feed (§ 2) |

| Decision    | Answer                                                                                | Why                                                                                         |
|-------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Stream      | <code>admin_logs_streaming</code> for live;<br><code>admin_logs</code> for backfill   | They are not interchangeable — different retention, ordering, and duplicate behaviour (§ 4) |
| Collector   | <b>Workflow</b> up to ~35,000 events per run; Python beyond that or for deep backfill | The 120-second JS-runtime timeout is the only real boundary (§ 6)                           |
| Destination | Log Ingestion API v2, dedicated <code>log.source</code>                               | No events-per-minute cap; 10 MB / 50,000 records per request (§ 9)                          |
| Retention   | Extract metrics, then discard raw by default; dedicated bucket for what you keep      | ~1.5 GB/day at 1M events is a real budget line (§ 5)                                        |

**Start with the signal that costs nothing.** If your own applications call Box, OneAgent is already tracing those calls. Query them before building anything — see § 3.

## 2. What Box Is — and Why There Is No Turnkey Integration

Box is a cloud content management platform — file storage, sharing, workflow, and governance — operated entirely by Box. In observability terms that single fact settles the architecture: there is no host to install OneAgent on, no process to inject, no infrastructure to discover. Everything you can learn about Box, you learn through its API.

This is the same shape as Zscaler (**FAQ-20**) and Adobe AEM as a Cloud Service (**FAQ-18**), and it is the pattern generalized in **FAQ-19**.

### 2.1 No Hub listing — verified, not assumed

As of August 2026 there is no Dynatrace Hub listing, extension, or first-party integration for Box. This was checked three independent ways — the Hub catalog, general web search, and the Dynatrace documentation assistant — and all three returned nothing.

Per the corpus rule that a negative claim needs a positive source: what is being reported here is the result of three catalog checks on 08/25/2026, not an inference from one empty

search. Re-verify before quoting this in a design document; Hub listings appear over time.

The practical consequence is ownership. There is no vendor-maintained connector to inherit, and no upgrade path that will fix your integration for you. That is a normal position — it is simply better decided deliberately in week one than discovered in week three.

## 2.2 Why webhooks are the wrong answer — and why people reach for them

Box does publish webhooks, which is exactly why this goes wrong. They are a real feature that does a different job.

| Box webhooks V2                                                                             | Consequence for an audit feed                                |
|---------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Attach to <b>specific files or folders</b> ; <b>cannot be set on the root folder</b> (ID 0) | There is no account-wide subscription to create              |
| Cap of <b>1,000 webhooks per application + user</b> combination                             | You could watch 1,000 folders — not an enterprise            |
| Triggers are content actions (upload, copy, move, share, delete)                            | <b>No logins, no admin actions, no SHIELD_ALERT</b>          |
| Delivery requires a 2xx within 30 s; Box retries 12× over 2 hours                           | A receiver outage drops events with no cursor to replay from |
| Token expiry degrades the payload to <b>NO_ACTIVE_SESSION</b>                               | Silent partial failure                                       |

Enterprise audit events are reachable only through `GET /events` with `stream_type` set to `admin_logs` or `admin_logs_streaming`, and Box's own documentation states that **"the enterprise event feed does not support long polling."** It is pull-only.

**If someone has recommended a webhook for this, they have answered a different question.** Webhooks are the right tool for reacting to changes in a specific high-value folder — a contract repository, a regulated dataset. They are structurally incapable of carrying the audit trail.

### Sources:

- [Webhooks \(Box Dev Docs\)](#)
- [Webhook limitations V2 \(Box Dev Docs\)](#)
- [Get Enterprise Events \(Box Dev Docs\)](#)

- [Box Hub catalog \(Dynatrace\)](#)
- **Derived:** § 2.1's no-listing claim is the result of three independent catalog checks on 08/25/2026, not a single search

## 3. The Four Signal Classes

FAQ-19 § 2 splits any third-party SaaS into signal classes and insists you choose which questions you are answering before choosing a transport. For Box there are four, and they differ enormously in cost.

### Box — Four Signal Classes

Start with the one that costs nothing. Only two of the four require a collector.

#### ZERO EFFORT

##### Your apps calling Box

Source:  
**OneAgent (already on)**

Answers:  
Latency and errors on outbound Box API calls from your own services

Query today:  
`span.kind == "client"`  
`server.address`

#### BUILD COLLECTOR

##### Audit events (enterprise)

Source:  
**GET /events (pull)**

Answers:  
Who did what — logins, downloads, shares, permission changes

Constraint:  
**No push. No webhooks.**  
**No long polling.**  
Cursor must persist.

#### FREE IF ABOVE

##### Shield alerts (security)

Source:  
**Same event stream**

Answers:  
Compromised accounts, anomalous downloads, malware, ransomware

Filter on:  
`event_type == "SHIELD_ALERT"`  
**Separate licence.**

#### LOW LIFT

##### Availability

Source:  
**Statuspage JSON + Synthetic monitor**

Answers:  
Is Box up (vendor view) vs. is Box up for us (your view)

Both matter — they disagree more often than you would expect.

#### Sequencing

Query signal 1 before building anything — if it answers “is Box hurting us”, the collector may not be needed. Signals 2 and 3 share one collector, so build it once.

### 3.1 Start with the signal that costs nothing

If any of your applications call the Box API, OneAgent is **already** capturing those as client spans. No Box configuration, no credentials, no collector. This is the fastest path to an answer and it is routinely skipped.

Run this before you build anything:

```
// Signal 1 – your own services calling Box. Requires no Box integration at all.
// Note the field names: server.address and url.full are the stable fields.
// http.url does NOT exist in the semantic dictionary and returns zero rows
// silently – see § 17. Same shape, different field: span.status_code values are
// LOWERCASE, so `== "ERROR"` reports a 0% failure rate for every address.
fetch spans, from:-24h
| filter span.kind == "client" and endsWith(server.address, "box.com")
| summarize {calls = count(),
              failures = countIf(span.status_code == "error"),
              p90_ms = percentile(duration, 90) / 1000000},
              by:{server.address}
| fieldsAdd failure_rate_pct = round(100.0 * failures / calls, decimals: 2)
| sort calls desc
```

If that returns rows, you already have Box performance telemetry and can build a dashboard today. If it returns nothing, confirm *why* before concluding your services do not call Box — an empty result here has at least three causes (no Box traffic, no OneAgent on the calling tier, or a mistyped field name), and only one of them is informative.

### 3.2 What each remaining signal is actually for

| Signal               | Answers                                                             | Does not answer                                  |
|----------------------|---------------------------------------------------------------------|--------------------------------------------------|
| <b>Audit events</b>  | Who accessed, shared, downloaded, or deleted what, when, from where | Whether Box was slow or erroring                 |
| <b>Shield alerts</b> | Which of those actions Box's own detection considers a threat       | Anything Shield is not licensed to detect        |
| <b>Availability</b>  | Whether Box is reachable and healthy — vendor-declared and observed | Which user was affected, or what they were doing |

The common mistake is expecting audit events to answer performance questions. They are an access record, not a latency record. Box does not publish per-request timing for its own service; if you need Box performance, it comes from signal 1 (your calls) or signal 4 (synthetics), never from the audit feed.

**Sources:** [Global field reference \(DT docs\)](#), [List user and enterprise events \(Box Dev Docs\)](#). **Derived:**

§ 3.1's field-name caution comes from executing both forms against a live tenant on 08/25/2026 —

`http.url` returned zero rows while scanning 9.8M records.

## 4. The Events API — The Constraint That Shapes Everything

Everything enterprise-wide comes from one endpoint, `GET /events`, in one of two mutually exclusive modes. Choosing between them is the first real design decision, and most teams need both.

|              | <code>admin_logs_streaming</code>     | <code>admin_logs</code>                   |
|--------------|---------------------------------------|-------------------------------------------|
| Retention    | 2 weeks                               | 1 year                                    |
| Latency      | Near real-time                        | Higher                                    |
| Ordering     | Out of chronological order            | Chronological                             |
| Duplicates   | Yes — dedupe on <code>event_id</code> | None                                      |
| Designed for | Live monitoring and alerting          | Historical query, backfill, investigation |

**Use streaming for the live pipeline and historical for backfill.** They are not alternatives to choose between once; they are two jobs. A collector that only runs streaming has a two-week memory and no way to recover from a longer outage. A collector that only runs historical is too slow to alert on.

### 4.1 The constraints that shape the collector

| Constraint | Value                                                                     | Design consequence                                        |
|------------|---------------------------------------------------------------------------|-----------------------------------------------------------|
| No push    | <i>"The enterprise event feed does not support long polling"</i>          | Scheduled pull. There is no event-driven option           |
| Page size  | 500 events maximum                                                        | Paging loop required; drives the sizing math in § 5       |
| Cursor     | <code>stream_position</code> in,<br><code>next_stream_position</code> out | <b>Must persist between runs</b> or you re-ingest or skip |

| Constraint | Value                                                             | Design consequence                                           |
|------------|-------------------------------------------------------------------|--------------------------------------------------------------|
| Rate limit | 1,000 requests/min per user; 429 returns <code>retry-after</code> | Exponential backoff. Rarely binding — see § 5                |
| Permission | Enterprise admin/co-admin with <i>"Run new reports..."</i>        | Missing it returns nothing, indistinguishable from no events |

## 4.2 The cursor is the whole design

Everything difficult about this collector reduces to one question: *where did I get to last time?* Box hands you `next_stream_position` with every page; you hand it back on the next call. Lose it and you either replay events you already have (paying twice) or skip events you never got (a compliance gap you cannot detect after the fact).

Because `admin_logs_streaming` retains only two weeks, a lost cursor is not always recoverable. This is the single highest-consequence property of the integration and it drives § 13.

### Sources:

- [Get Enterprise Events \(Box Dev Docs\)](#)
- [List user and enterprise events \(Box Dev Docs\)](#)
- [Rate Limits \(Box Dev Docs\)](#)
- [New Enterprise Event Stream API \(Box Support\)](#)

## 5. Sizing: A Worked Example at 1,000,000 Events/Day

This section works a real number end to end, because the volume question is usually asked about the wrong endpoint. A migrating customer asked whether a webhook could handle 1M Box events per day; the answer they received addressed only the Dynatrace ingest limits. Both halves matter, and — as it turns out — **neither is the constraint**.

## 5.1 Both endpoints have ample headroom

| Side                    | Documented limit                                                       | At 1M events/day                                             | Verdict          |
|-------------------------|------------------------------------------------------------------------|--------------------------------------------------------------|------------------|
| <b>Dynatrace ingest</b> | <b>No events-per-minute cap</b> ; 10 MB and 50,000 records per request | ~11.6 events/sec                                             | Not a constraint |
| <b>Box Events API</b>   | 500 events/page; 1,000 requests/min per user                           | $1M \div 500 = 2,000$ pages/day $\approx$ <b>1.4 req/min</b> | ~0.1% of budget  |

1,000,000 events per day is a modest load for both systems. **The bottleneck is neither endpoint — it is that no collector exists between them.** That is the part a question about "which ingestion option handles the volume" does not surface, because the ingest API is the destination, not the connection.

## 5.2 What that means for a Workflow

Polling every 5 minutes gives 288 runs/day and ~3,472 events per run — about **7 pages of 500**. Against a 120-second execution ceiling that is comfortable:

| Quantity                      | Value        | Basis                                                         |
|-------------------------------|--------------|---------------------------------------------------------------|
| Runs per day (5-min schedule) | 288          | Schedule                                                      |
| Events per run                | ~3,472       | $1M \div 288$                                                 |
| Pages per run                 | ~7           | 500 events/page                                               |
| Estimated time per run        | ~15 s        | ~1–2 s per page — <b>an estimate, not a documented figure</b> |
| Execution ceiling             | <b>120 s</b> | JS-runtime hard limit                                         |
| Approximate headroom used     | <b>~10%</b>  | Derived                                                       |

Mark the page-latency assumption clearly and measure it in your own tenant. The documented numbers here — 120 s, 500 events/page, 1,000 req/min, no ingest rate cap — are verified. The per-page round-trip time is not, and it is the one input that could move the conclusion.

## 5.3 The cost line nobody quotes

At roughly 1.5 KB per event:

|           | Volume  |
|-----------|---------|
| Per day   | ~1.5 GB |
| Per month | ~45 GB  |

That is a real DPS line item, and it arrives whether or not anyone budgeted for it. It is also the strongest argument for the extract-then-discard posture in § 9: most of those 45 GB are `PREVIEW` and `DOWNLOAD` events whose analytical value is fully captured by a counter.

Decide retention *before* you turn the collector on. Reducing retention later does not refund what you already ingested.

**Sources:** [Log ingestion limits \(DT docs\)](#), [Rate Limits \(Box Dev Docs\)](#), [Get Enterprise Events \(Box Dev Docs\)](#). **Derived:** § 5.1–5.3 combine the documented per-side limits with a 1M/day rate and a ~1.5 KB average event size; the per-page latency and event size are estimates to verify locally.

## 6. Choosing the Collector — Workflow or Python

### Choosing the Collector — Workflow or Python

The 120-second JS-runtime timeout is the whole decision. Everything else is preference.

#### Dynatrace Workflow (JS runtime)

**No external infrastructure to run or secure**

Secrets in Credential Vault — never in code  
Cursor via stateClient; ingest via logsClient.storeLog  
Scheduling, retry and failure alerting are built in

**Hard limits: 120 s run · 256 MB · 5 MB output**  
Must be a STANDARD workflow — automation-utils returns 404 on simple workflows

#### Python collector

**No execution ceiling — backfill and replay at will**

pip install boxsdk — v10+, Client Credentials Grant  
Cursor in your own durable store  
Run on K8s CronJob, Lambda, or any scheduler

**You own runtime, secrets, scheduling and alerting**  
box-sdk-gen is the DEPRECATED package — the name suggests otherwise. Use boxsdk.

#### Where 1M events/day actually lands

Polling every 5 minutes = 288 runs/day → ~3,472 events per run → ~7 pages of 500.

~7 pages

remaining headroom inside the same 120 s run — roughly 63 more pages

1M events/day sits here — about 10% of the run budget

120 s ceiling

**Verdict at 1M/day: use a Workflow.**

Reach for Python when you need backfill beyond the two-week window, or when a single run would exceed ~35,000 events. Page latency is an estimate — measure it in your own tenant before relying on the margin.

|                                     | Workflow (JS runtime)              | Python                            |
|-------------------------------------|------------------------------------|-----------------------------------|
| Infrastructure                      | None — runs inside Dynatrace       | You provision, run, patch, secure |
| Execution ceiling                   | 120 s per run (hard)               | None                              |
| Memory / output                     | 256 MB / 5 MB output               | Yours                             |
| Secrets                             | Credential Vault                   | Your own secret store             |
| Cursor                              | <code>stateClient</code> app state | Your own durable store            |
| Scheduling, retry, failure alerting | Built in                           | You build all three               |
| Backfill over months                | Impractical                        | Natural fit                       |

**At 1M events/day, use a Workflow.** It clears the ceiling with roughly 90% headroom (§ 5.2) and removes an entire class of operational work — there is no container to keep patched, no secret to rotate outside Dynatrace, and failure alerting is already there.

#### Reach for Python when:

- You need to backfill more than the two-week streaming window, or replay a year of `admin_logs`.
- Sustained volume pushes a single run past roughly 35,000 events (~70 pages).
- Policy requires the collector to live in your own estate.
- You want to enrich against systems the JS runtime cannot reach.

A common and sensible split: **Python for the one-time historical backfill, Workflow for steady state.** They write to the same `log.source` and the same bucket, and the backfill runs once.

**Sources:** [JavaScript runtime limits \(Dynatrace Developer\)](#), [Get Enterprise Events \(Box Dev Docs\)](#).

**Derived:** the ~35,000-events-per-run boundary combines the documented 120 s ceiling with the 500-event page size and an estimated per-page latency — verify locally per § 5.2.

## 7. Implementation A — Dynatrace Workflow

The recommended path at typical volume. No infrastructure, secrets stay in the Credential Vault, and scheduling plus failure alerting come free.

### 7.1 Three things to set up first

| Step                             | What                                                                                                    | Why it matters                                                                                                                               |
|----------------------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Credential Vault entry</b> | Store the Box <code>client_id</code> / <code>client_secret</code> as a username/password credential     | Never put Box secrets in workflow code — they are readable by anyone who can view the workflow                                               |
| <b>2. Outbound allowlist</b>     | Add <code>api.box.com</code> to <code>builtin:dt-javascript-runtime.allowed-outbound-connections</code> | <b>A non-allowlisted host fails silently.</b> This is the single most common "my workflow does nothing" cause                                |
| <b>3. Standard workflow</b>      | Create a <b>standard</b> workflow, not a simple one                                                     | <code>@dynatrace-sdk/automation-utils</code> returns <b>404</b> on simple workflows. Standard workflows also bill differently — see ALERT-03 |

### 7.2 The collector

A single **Run JavaScript** action on a 5-minute schedule. It authenticates, reads the cursor, pages until drained, ingests, and writes the cursor back.

```

import { credentialVaultClient, logsClient } from "@dynatrace-sdk/client-
classic-environment-v2";
import { stateClient } from "@dynatrace-sdk/client-state";

const BOX_CREDENTIAL_ID = "CREDENTIALS_VAULT-XXXXXXXXXXXXXXXXXX";
const BOX_ENTERPRISE_ID = "1234567";
const CURSOR_KEY      = "box-events-cursor";
const LOG_SOURCE       = "box.events";
const PAGE_LIMIT       = 500;
const MAX_PAGES        = 60;          // stay clear of the 120 s ceiling
const INGEST_BATCH     = 1000;       // well under 50,000 records / 10 MB

export default async function () {
  // --- 1. Authenticate (Client Credentials Grant) -----
  const creds = await credentialVaultClient.getCredentialsDetails({
    id: BOX_CREDENTIAL_ID,
  });

  const tokenRes = await fetch("https://api.box.com/oauth2/token", {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body: new URLSearchParams({
      grant_type: "client_credentials",
      client_id: creds.username,
      client_secret: creds.password,
      box_subject_type: "enterprise",
      box_subject_id: BOX_ENTERPRISE_ID,
    }).toString(),
  });

  if (!tokenRes.ok) {
    // Fail loudly. A silent auth failure looks exactly like "no events".
    throw new Error(`Box auth failed: ${tokenRes.status} ${await
tokenRes.text()}`);
  }
  const { access_token } = await tokenRes.json();

  // --- 2. Read the cursor -----
  // "now" on first run starts at the live edge. Use 0 to replay everything
  // the streaming window still holds (up to two weeks).
  let streamPosition = "now";
  try {
    const saved = await stateClient.getAppState({ key: CURSOR_KEY });
    if (saved?.value) streamPosition = saved.value;
  } catch (e) {
    // No cursor yet – first run. Any other error should surface.
    if (e?.status !== 404) throw e;
  }
}

```

```

}

// --- 3. Page until drained -----
const seen = new Set();
const records = [];
let pages = 0;

while (pages < MAX_PAGES) {
  const url =
    `https://api.box.com/2.0/events?stream_type=admin_logs_streaming` +
    `&limit=${PAGE_LIMIT}&stream_position=${encodeURIComponent(streamPosition)}`;

  const res = await fetch(url, {
    headers: { Authorization: `Bearer ${access_token}` },
  });

  if (res.status === 429) {
    // Respect Box's backoff and finish on the next scheduled run.
    break;
  }
  if (!res.ok) {
    throw new Error(`Box events failed: ${res.status} ${await res.text()}`);
  }

  const page = await res.json();
  const entries = page.entries ?? [];

  for (const evt of entries) {
    // admin_logs_streaming can repeat events – dedupe within the run.
    if (seen.has(evt.event_id)) continue;
    seen.add(evt.event_id);

    records.push({
      timestamp: evt.created_at,
      content: JSON.stringify(evt),
      "log.source": LOG_SOURCE,
      "box.event_id": evt.event_id,
      "box.event_type": evt.event_type,
      "box.user.login": evt.created_by?.login,
      "box.item.name": evt.source?.item_name,
      "box.ip_address": evt.ip_address,
    });
  }

  streamPosition = page.next_stream_position;
  pages++;
}

```

```

    // A short page means the stream is drained.
    if (entries.length < PAGE_LIMIT) break;
}

// --- 4. Ingest in batches -----
for (let i = 0; i < records.length; i += INGEST_BATCH) {
    await logsClient.storeLog({
        type: "application/json; charset=utf-8",
        body: records.slice(i, i + INGEST_BATCH),
    });
}

// --- 5. Persist the cursor LAST -----
// Only after a successful ingest. Writing it earlier means a failed ingest
// silently loses every event in this batch.
await stateClient.setAppState({
    key: CURSOR_KEY,
    body: { value: String(streamPosition) },
});

return { pages, ingested: records.length, cursor: streamPosition };
}

```

### 7.3 Four details in that code that are load-bearing

| Detail                          | Why                                                                                                                                                                                    |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cursor written last</b>      | If ingest fails after the cursor advances, those events are gone — and with a two-week window they may be unrecoverable. Write-after-success makes a failed run replay instead of lose |
| <b>No TTL on the cursor</b>     | <code>stateClient TTL ( validUntilTime )</code> accepts <code>now+1m ... now+90d</code> . Set one and your cursor silently expires, resetting the collector. <b>Omit it</b>            |
| <b>MAX_PAGES guard</b>          | Bounds the run below the 120 s ceiling. A backlog drains across several runs rather than timing out forever on the first                                                               |
| <b>Throwing on auth failure</b> | An unhandled non-OK response would leave you with an empty ingest that looks identical to a quiet Box tenant                                                                           |

### 7.4 Alert on the collector, not just on Box

A Workflow that stops running produces no logs — and no logs looks exactly like no activity. Add a second workflow that runs the § 13 staleness query and pages someone

when it goes stale. **This is not optional at a two-week retention window.**

**Sources:**

- [JavaScript runtime limits \(Dynatrace Developer\)](#)
- [client-state SDK \(Dynatrace Developer\)](#)
- [Credential vault client \(Dynatrace Developer\)](#)
- [Client Credentials Grant \(Box Dev Docs\)](#)
- [List user and enterprise events \(Box Dev Docs\)](#)
- **Derived:** § 7.3's write-cursor-last rule combines the documented two-week streaming retention with the absence of any replay mechanism once a cursor advances

## 8. Implementation B — Python

Use Python for historical backfill, for replay, or when sustained volume exceeds what a single 120-second run can drain.

### 8.1 Install the right SDK — this one is easy to get backwards

```
pip install boxsdk requests
```

**boxsdk (v10+)** is the current Box Python SDK. **box-sdk-gen** is the deprecated one. The naming is actively misleading — "gen" sounds like the newer generation, and it was, briefly: it shipped as a standalone package and was deprecated on **17 September 2025** when its functionality was absorbed into the core SDK at v10.

The PyPI release history settles it: **boxsdk** 10.14.0 shipped 05 August 2026, while **box-sdk-gen** has not released since 05 September 2025.

## 8.2 The collector

```
"""Poll the Box enterprise event stream and ship to Dynatrace.

Steady state: --stream-type admin_logs_streaming
Backfill:     --stream-type admin_logs --created-after 2026-01-01T00:00:00Z
"""

import argparse
import json
import os
import time
from pathlib import Path

import requests
from boxsdk import Client, CCGAuth

BOX_CLIENT_ID      = os.environ["BOX_CLIENT_ID"]
BOX_CLIENT_SECRET  = os.environ["BOX_CLIENT_SECRET"]
BOX_ENTERPRISE_ID  = os.environ["BOX_ENTERPRISE_ID"]

DT_ENV_URL        = os.environ["DT_ENV_URL"]          #
https://abc12345.live.dynatrace.com
DT_API_TOKEN      = os.environ["DT_API_TOKEN"]        # scope: logs.ingest

CURSOR_FILE       = Path(os.environ.get("BOX_CURSOR_FILE", "/var/lib/box/cursor"))
LOG_SOURCE        = "box.events"
PAGE_LIMIT        = 500
INGEST_BATCH      = 1000      # well under the 50,000-record / 10 MB request limits


def box_client() -> Client:
    """Server-side auth. No user interaction, no refresh-token juggling."""
    auth = CCGAuth(
        client_id=BOX_CLIENT_ID,
        client_secret=BOX_CLIENT_SECRET,
        enterprise_id=BOX_ENTERPRISE_ID,
    )
    return Client(auth)


def read_cursor(default: str = "now") -> str:
    return CURSOR_FILE.read_text().strip() if CURSOR_FILE.exists() else
default


def write_cursor(position: str) -> None:
    """Write atomically – a torn cursor file is worse than no cursor file."""
```

```
CURSOR_FILE.parent.mkdir(parents=True, exist_ok=True)
tmp = CURSOR_FILE.with_suffix(".tmp")
tmp.write_text(str(position))
tmp.replace(CURSOR_FILE)
```

```
def to_dt_record(evt: dict) -> dict:
    """Flatten a Box event into a Dynatrace log record."""
    created_by = evt.get("created_by") or {}
    source      = evt.get("source") or {}
    return {
        "timestamp":      evt.get("created_at"),
        "content":         json.dumps(evt, separators=(",", ":")),
        "log.source":     LOG_SOURCE,
        "box.event_id":   evt.get("event_id"),
        "box.event_type": evt.get("event_type"),
        "box.user.login": created_by.get("login"),
        "box.item.name":   source.get("item_name"),
        "box.item.type":   source.get("type"),
        "box.ip_address":  evt.get("ip_address"),
    }

def ingest(records: list[dict]) -> None:
    """POST to the Dynatrace Log Ingestion API, batched."""
    url = f"{DT_ENV_URL}/api/v2/logs/ingest"
    headers = {
        "Authorization": f"Api-Token {DT_API_TOKEN}",
        "Content-Type": "application/json; charset=utf-8",
    }
    for i in range(0, len(records), INGEST_BATCH):
        batch = records[i:i + INGEST_BATCH]
        for attempt in range(5):
            resp = requests.post(url, headers=headers,
                                data=json.dumps(batch), timeout=30)
            if resp.status_code < 300:
                break
            # 413 means the batch expanded past 10 MB (or 16 MB after
            # preprocessing) – halving is the fix, not retrying.
            if resp.status_code == 413 and len(batch) > 1:
                mid = len(batch) // 2
                ingest(batch[:mid])
                ingest(batch[mid:])
                break
            if resp.status_code in (429, 500, 502, 503, 504):
                time.sleep(2 ** attempt)
                continue
            resp.raise_for_status()
```

```

        else:
            raise RuntimeError("Dynatrace ingest failed after 5 attempts")

def collect(stream_type: str, created_after: str | None = None) -> int:
    client = box_client()
    position = read_cursor()
    seen: set[str] = set()
    total = 0

    while True:
        params = {
            "stream_type": stream_type,
            "limit": PAGE_LIMIT,
            "stream_position": position,
        }
        if created_after:
            params["created_after"] = created_after

        # The SDK handles auth, retry and token refresh for us.
        page = client.make_request(
            "GET", client.get_url("events"), params=params
        ).json()

        entries = page.get("entries", [])
        if not entries:
            break

        records = []
        for evt in entries:
            eid = evt.get("event_id")
            # admin_logs_streaming repeats events; admin_logs does not.
            if eid in seen:
                continue
            seen.add(eid)
            records.append(to_dt_record(evt))

        if records:
            ingest(records)
            total += len(records)

        position = page.get("next_stream_position")
        # Persist only after a successful ingest.
        write_cursor(position)

        if len(entries) < PAGE_LIMIT:
            break

```

```

    return total

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--stream-type", default="admin_logs_streaming",
                    choices=["admin_logs_streaming", "admin_logs"])
    ap.add_argument("--created-after", default=None,
                    help="RFC3339 timestamp; admin_logs backfill only")
    args = ap.parse_args()

    count = collect(args.stream_type, args.created_after)
    print(f"ingested {count} events")

```

## 8.3 Running it

| Mode                | Command                                                                                            | Notes                                                                            |
|---------------------|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Steady state</b> | <code>python box_collector.py</code>                                                               | Every 5 min via cron, K8s <code>CronJob</code> , or a systemd timer              |
| <b>Backfill</b>     | <code>python box_collector.py --stream-type admin_logs --created-after 2026-01-01T00:00:00Z</code> | Run once. Use a <b>separate cursor file</b> so it cannot clobber the live cursor |

**Keep the two cursors apart.** A backfill sharing the live cursor will drag the streaming collector backwards or forwards unpredictably — the most common way this design corrupts itself.

## 8.4 Where to run it

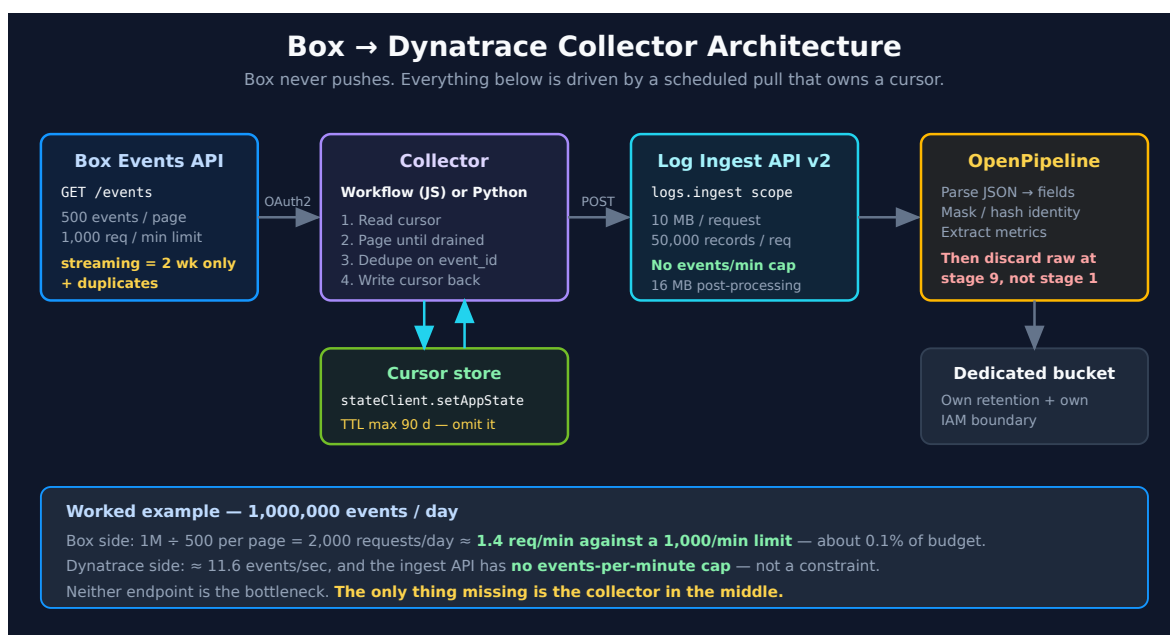
| Host                                   | Fit                                                                                       |
|----------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Kubernetes</b> <code>CronJob</code> | Best fit if you already run K8s — secrets, scheduling, and restarts are solved            |
| <b>AWS Lambda / Azure Functions</b>    | Fine; keep the cursor in DynamoDB, Table Storage, or a parameter store rather than a file |
| <b>A VM with cron</b>                  | Works. Ensure the cursor file is on persistent storage, not ephemeral disk                |

Whatever runs it needs monitoring of its own — see § 13.

#### Sources:

- [boxsdk on PyPI](#)
- [Deprecated Box Next Gen Python SDK \(Box Dev Docs\)](#)
- [Client Credentials Grant \(Box Dev Docs\)](#)
- [POST ingest logs \(DT docs\)](#)
- [Log ingestion limits \(DT docs\)](#)
- **Derived:** § 8.1's current-vs-deprecated verdict comes from PyPI release dates checked 08/25/2026 against Box's deprecation notice

## 9. Ingestion, Processing, and Storage



### 9.1 Ingest limits that actually bind

| Limit               | Value           | Consequence                                |
|---------------------|-----------------|--------------------------------------------|
| Events per minute   | <b>No limit</b> | Volume is not the problem (§ 5)            |
| Payload per request | <b>10 MB</b>    | Batch by size, not only by count           |
| Records per request | <b>50,000</b>   | Rarely the binding limit — size hits first |

| Limit                     | Value                       | Consequence                                                        |
|---------------------------|-----------------------------|--------------------------------------------------------------------|
| Post-processing expansion | Rejected above <b>16 MB</b> | A batch inside 10 MB can still fail after processing. Halve on 413 |
| Attributes per record     | 500                         | Ample for Box events                                               |
| Attribute value length    | 32 kB                       | A large <code>additional_details</code> can approach this          |

## 9.2 Extract, then discard — in that order

The mechanics and the trap are in **FAQ-19 § 4**; do not configure this without reading it. In short:

| Extract               | Example                                                                                        | Retention posture                                                             |
|-----------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Metric</b>         | Event counts by <code>event_type</code> and user; download volume by user; failed-login counts | <b>Keep the metric.</b> Discard the source record unless separately justified |
| <b>Business event</b> | A file in a regulated folder was shared externally                                             | <b>Keep</b> , minimal fields                                                  |
| <b>Alert</b>          | <code>SHIELD_ALERT</code> (§ 11)                                                               | <b>Keep</b> with full context                                                 |
| <b>Raw record</b>     | Every <code>PREVIEW</code> and <code>DOWNLOAD</code> event                                     | <b>Discard by default.</b> This is the bulk of your 45 GB/month               |

**The trap, stated once more because it silently defeats the whole design:** `Drop record` at **stage 1** pre-empts every extractor downstream — your metrics never get built. Use `No storage assignment` at **stage 9** instead. Extract first, discard last.

**Field-level controls in the Processing stage, before storage:**

| Field class                                                     | Control                                                                                                                                                                          |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>created_by.login</code> ,<br><code>created_by.name</code> | <b>Hash</b> where the analysis only needs correlation, not identity. Note a hash of a corporate email is enumerable — a control against casual exposure, not a privacy guarantee |
| <code>source.item_name</code>                                   | <b>Consider masking.</b> File names leak project names, client names, and deal names                                                                                             |

| Field class                                                | Control                                             |
|------------------------------------------------------------|-----------------------------------------------------|
| <code>ip_address</code>                                    | Keep — needed for the § 11 and § 12 access analysis |
| <code>additional_details</code> on high-volume event types | <b>Remove.</b> Cheapest control, apply first        |

## 9.3 Bucket isolation

Put Box events in a **dedicated bucket**. Two independent reasons, either sufficient:

- **Retention differs.** An audit trail is typically kept far longer than application logs, and paying application-log retention across 45 GB/month of Box events is expensive.
- **Access differs.** This feed names individuals and the documents they touched. It should be readable by security and compliance, not by everyone who can read application logs. A bucket is the boundary that makes that enforceable — see **ORGNZ**.

### Sources:

- [Log ingestion limits \(DT docs\)](#)
- [Processing in OpenPipeline \(DT docs\)](#)
- [Configure data storage and retention for logs \(DT docs\)](#)
- [Log ingestion \(DT docs\)](#) — the ingestion-route overview this section's API choice sits inside

## 10. Field Mapping Starting Point

Box events are JSON, so the parse is mechanical. What matters is normalizing to names that read the same as the rest of your data.

| Box field               | Suggested Dynatrace field   | Notes                                                                                                                                                            |
|-------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>event_id</code>   | <code>box.event_id</code>   | <b>Dedupe key.</b> Required — streaming repeats events                                                                                                           |
| <code>event_type</code> | <code>box.event_type</code> | Primary dimension. <code>LOGIN</code> , <code>UPLOAD</code> , <code>DOWNLOAD</code> , <code>SHARE</code> , <code>DELETE</code> , <code>SHIELD_ALERT</code> , ... |

| Box field                       | Suggested Dynatrace field   | Notes                                                                               |
|---------------------------------|-----------------------------|-------------------------------------------------------------------------------------|
| <code>created_at</code>         | <code>timestamp</code>      | Map to the record timestamp, not a custom field                                     |
| <code>created_by.login</code>   | <code>box.user.login</code> | Consider hashing (§ 9.2)                                                            |
| <code>created_by.id</code>      | <code>box.user.id</code>    | Stable across renames — prefer for joins                                            |
| <code>created_by.name</code>    | <code>box.user.name</code>  | Display only                                                                        |
| <code>source.type</code>        | <code>box.item.type</code>  | <code>file</code> , <code>folder</code> , <code>user</code> , <code>app_item</code> |
| <code>source.id</code>          | <code>box.item.id</code>    | Stable identifier                                                                   |
| <code>source.item_name</code>   | <code>box.item.name</code>  | Consider masking (§ 9.2)                                                            |
| <code>ip_address</code>         | <code>box.ip_address</code> | Retain — needed for access analysis                                                 |
| <code>additional_details</code> | <code>box.details.*</code>  | Enterprise-stream only. Shape varies by <code>event_type</code>                     |

**Normalize before you extract.** Metric dimensions and any Smartscape identifiers are computed from fields as they stand at their stage, so renaming after extraction leaves metrics carrying the old dimension names. Rename in Processing, then extract — the same ordering rule as § 9.2.

If you also ingest identity events from elsewhere (Okta, Entra ID), map `box.user.login` onto whatever principal field those feeds use. Correlating a Box download with the sign-in that preceded it is one of the highest-value things this integration enables, and it only works if the principal is spelled the same way in both.

## 11. Box Shield Alerts

Box Shield is Box's own threat detection. If it is licensed, its alerts arrive **in the same event stream** you are already collecting — no second integration, no extra credential. If it is not licensed, no `SHIELD_ALERT` events exist and nothing below applies.

## 11.1 What Shield detects

| Alert type           | Trigger                                                               |
|----------------------|-----------------------------------------------------------------------|
| Suspicious Locations | Access from unusual or excluded geographies                           |
| Suspicious Sessions  | Unusual user-agents, new IPs, implausible travel                      |
| Anomalous Downloads  | Download behaviour consistent with data theft                         |
| Malicious Content    | Malware detected on upload                                            |
| Ransomware Activity  | File extensions consistent with ransomware ( <b>Shield Pro only</b> ) |

## 11.2 The alert payload

Shield alerts follow the standard event schema with `event_type == "SHIELD_ALERT"`, carrying an `additional_details.shield_alert` object:

| Field                                                  | Use                                                                        |
|--------------------------------------------------------|----------------------------------------------------------------------------|
| <code>rule_category</code> ,<br><code>rule_name</code> | Classification — the primary dimension for routing                         |
| <code>risk_score</code>                                | Numeric severity; good threshold for problem creation                      |
| <code>priority</code>                                  | Box's own severity label                                                   |
| <code>alert_summary</code>                             | Context: IPs, affected files, timestamps                                   |
| <code>link</code>                                      | Deep link back to the Shield console — <b>put this in the notification</b> |
| <code>rule_response_action</code>                      | What Shield already did (e.g. terminated the session)                      |

The `link` field deserves emphasis. An alert that reaches an analyst without a route back to the vendor console costs several minutes of navigation per incident. Carry it through processing into the notification payload.

## 11.3 Routing them

Shield alerts are the one Box signal that should page someone. Route them per **FAQ-21**:

1. Extract `SHIELD_ALERT` events in OpenPipeline into a **davis event** or business event.

2. Attach ownership so the trigger can route on it — FAQ-21 covers why `dt.owner` on an event needs an explicit, non-retroactive Problem fields mapping.
3. Trigger a **problem-triggered workflow**, filtering on `rule_category` and `risk_score`.
4. **Keep these records.** This is the category where raw retention is justified — § 9.2's discard-by-default posture explicitly exempts it.

```
// Shield alert extraction. Runs standalone against a synthetic record – zero
// bytes scanned. Replace the `data record(...)` line with:
//  fetch logs, from:-24h | filter log.source == "box.events"
//  once your collector is running.
data record(content = "
{\"event_id\": \"a1\", \"event_type\": \"SHIELD_ALERT\", \"created_at\": \"2026-08-
25T10:15:02-07:00\", \"created_by\":
{\"login\": \"jdoe@example.com\"}, \"additional_details\": {\"shield_alert\":
{\"rule_category\": \"Anomalous Download\", \"rule_name\": \"Bulk
download\", \"risk_score\": 78, \"priority\": \"high\", \"alert_id\": \"9911\"}}}
)
| parse content, "JSON:evt"
| filter evt[event_type] == "SHIELD_ALERT"
| fieldsAdd box.alert.category    = evt[additional_details][shield_alert]
[rule_category],
                box.alert.risk_score = evt[additional_details][shield_alert]
[risk_score],
                box.alert.priority   = evt[additional_details][shield_alert]
[priority],
                box.user              = evt[created_by][login]
| fields box.alert.category, box.alert.risk_score, box.alert.priority,
box.user
```

**Sources:** [Shield Alert Events \(Box Dev Docs\)](#), [Box Shield \(Box\)](#). **Derived:** § 11.3's routing sequence applies FAQ-21's four-axis model to the Shield payload; the recommendation to carry `Link` into the notification is an operational inference, not a documented requirement.

## 12. Querying Box Events in Grail

Each query below runs standalone at **zero bytes scanned**. Swap the `data record(...)` line for `fetch logs, from:-24h | filter log.source == "box.events"` once data flows.

## 12.1 Parse a raw Box event

If you ingested `content` as the raw JSON (as both collectors in § 7 and § 8 do), this is how you get at the fields at query time. Doing it in OpenPipeline instead is cheaper — see § 9.2 — but this form is useful while validating.

```
// Parse a raw Box event record into queryable fields.
// Runs standalone – zero bytes scanned.
data record(content = "{\"event_id\":\"f82c314e-9a01-4f2b-b6d2-1c7a55e0d311\", \"event_type\":\"LOGIN\", \"created_at\":\"2026-08-25T10:15:02-07:00\", \"created_by\":{\"type\":\"user\", \"id\":\"12345\", \"name\":\"Jane Doe\", \"login\":\"jdoe@example.com\"}, \"source\":{\"type\":\"user\", \"id\":\"12345\"}, \"ip_address\":\"203.0.113.42\"}")
| parse content, "JSON:evt"
| fieldsAdd box.event_id = evt[event_id],
           box.event_type = evt[event_type],
           box.user       = evt[created_by][login],
           box.ip         = evt[ip_address]
| fields box.event_id, box.event_type, box.user, box.ip
```

## 12.2 Verify deduplication is working

`admin_logs_streaming` returns duplicates by design. At 1M events/day an unnoticed duplicate rate is a direct and recurring cost. Run this weekly.

```
// Duplicate rate on the Box feed. Anything above ~0 means dedupe is not
// working – check that the collector's `seen` set spans the whole run.
// Runs standalone against synthetic records – zero bytes scanned.
data record(box.event_id = "evt-001", box.event_type = "LOGIN"),
  record(box.event_id = "evt-001", box.event_type = "LOGIN"),
  record(box.event_id = "evt-002", box.event_type = "DOWNLOAD"),
  record(box.event_id = "evt-003", box.event_type = "SHIELD_ALERT")
| summarize {ingested = count(), unique = countDistinct(box.event_id)}
| fieldsAdd duplicates = ingested - unique
| fieldsAdd duplicate_pct = round(100.0 * duplicates / ingested, decimals: 1)
```

## 12.3 Confirm records landed in the right bucket

A bucket misassignment is silent — the data arrives, is queryable, and quietly inherits the wrong retention and the wrong access boundary.

```
// Which bucket are Box records actually in? Expect your dedicated bucket,  
// not `default_logs`. An empty result means no Box data yet – not success.  
fetch logs, from:-24h  
| filter log.source == "box.events"  
| summarize records = count(), by:{dt.system.bucket}  
| sort records desc
```

## 12.4 Daily volume against the sizing assumption

Confirm reality matches the § 5 budget. A sustained overshoot is the earliest warning that retention posture needs revisiting.

```
// Daily Box event volume vs. a 1M/day planning assumption.  
// Use interval:24h – interval:1d raises a calendar-duration warning.  
fetch logs, from:-7d  
| filter log.source == "box.events"  
| makeTimeseries events = count(), interval:24h  
| fieldsAdd daily_avg = arrayAvg(events)  
| fieldsAdd pct_of_1m_budget = round(100.0 * arrayAvg(events) / 1000000,  
decimals: 1)
```

---

## 13. Operating the Collector

This section is not optional, and it is the one most often skipped.

**The failure mode:** the collector stops. It produces no logs. No logs is indistinguishable from a quiet Box tenant. Nobody notices. Two weeks later the `admin_logs_streaming` window has rolled past the gap and **those events are permanently unrecoverable** — no replay, no backfill, no support ticket that recovers them.

Because Box retains the streaming feed for only two weeks, your detection window for a collector outage is *shorter than two weeks*. Alert on it.

### 13.1 The staleness query — and the bug in the obvious version

The natural way to write this check has a defect worth seeing, because it fails in the exact direction that hurts.

```
| fieldsAdd collector_status = if(minutes_since_last > 15, "STALLED", else: "OK")
```

When the collector has **never** run, `last_event` is null, so `now() - last_event` is null, `round(null)` is null, and `null > 15` evaluates false — landing on **"OK"**. The health check reports healthy precisely when there is no data at all. Verified against a live tenant on 08/25/2026: this form returned `collector_status: "OK"` alongside `events: 0`.

Test the record count explicitly:

```
// Collector health. Note the explicit events == 0 branch: without it, a
// collector that has NEVER run reports "OK", because null > 15 is false.
// Verified against a live tenant 08/25/2026.
fetch logs, from:-24h
| filter log.source == "box.events"
| summarize {last_event = takeMax(timestamp), events = count()}
| fieldsAdd minutes_since_last = round((now() - last_event) / 1m, decimals: 1)
| fieldsAdd collector_status = if(events == 0, "NO DATA – collector never ran
or is misconfigured",
                                else: if(minutes_since_last > 15, "STALLED",
                                else: "OK"))
```

## 13.2 What to alert on

| Condition                                              | Severity | Why                                                                       |
|--------------------------------------------------------|----------|---------------------------------------------------------------------------|
| No Box events for <b>&gt; 15 min</b>                   | High     | At a 5-min schedule, three missed runs is a real outage                   |
| No Box events <b>ever</b> ( <code>events == 0</code> ) | High     | Misconfiguration — usually the outbound allowlist or the admin permission |
| Duplicate rate rising above ~0                         | Medium   | Dedupe broken; you are paying twice                                       |
| Daily volume deviating sharply from baseline           | Medium   | Either a Box-side change or a partial collection failure                  |
| Workflow execution failure                             | High     | Route via <b>WFLOW</b> — it is already built in                           |

## 13.3 Recovering from an outage

| Outage length                  | Recovery                                                                                                                                                 |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| < 2 weeks                      | Reset the cursor to <code>0</code> and re-run. The streaming window still holds the events; dedupe on <code>event_id</code> protects against the overlap |
| > 2 weeks                      | Streaming cannot help. Backfill with <code>admin_logs</code> and <code>created_after</code> (§ 8.3) — up to one year, chronological and duplicate-free   |
| Cursor lost, collector healthy | Same as above: pick the stream by how far back the gap goes                                                                                              |

The existence of a one-year `admin_logs` backfill path is exactly why § 8's Python collector is worth having even in a Workflow-first design. **Build it before you need it** — writing a backfill tool during an active compliance gap is not when you want to be learning the API.

**Sources:** [Get Enterprise Events \(Box Dev Docs\)](#). **Derived:** § 13.1's null-comparison behaviour was observed directly by executing both query forms against a live tenant on 08/25/2026.

## 14. Availability Monitoring

Two different questions, two different answers, and they disagree more often than you would expect.

| Question          | Source                                                 | What it tells you                                         |
|-------------------|--------------------------------------------------------|-----------------------------------------------------------|
| Is Box up?        | <code>https://status.box.com/api/v2/status.json</code> | Box's own declaration, per component                      |
| Is Box up for us? | Dynatrace Synthetic HTTP monitor                       | Reachability and latency from your locations and networks |

## 14.1 The Box status feed

Box runs a Statuspage instance with a public JSON API — verified reachable and well-formed on 08/25/2026:

| Endpoint                          | Returns                                                                                                               |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>/api/v2/status.json</code>  | Overall indicator and description                                                                                     |
| <code>/api/v2/summary.json</code> | Per-component status — Content API, Box Web Application, Login/SSO, Uploads/Downloads, Box Platform / API, and others |

`summary.json` is the more useful of the two: "Box is up" is far less actionable than "the Content API is degraded", especially when your integration depends on exactly that component.

Poll it from the same Workflow that collects events — it is one extra `fetch` and requires no authentication.

## 14.2 Synthetic monitoring — when it adds something

A vendor status page reports what the vendor has decided to declare, on the vendor's timeline. A synthetic monitor reports what your users would actually experience. Add synthetics when:

- Box is on a critical business path and you need independent confirmation before opening a vendor ticket.
- You need per-location or per-network detail — a problem in one office is invisible to a global status page.
- You want Box availability in an SLO (§ 15) and need a signal you control the measurement of.

Point an HTTP monitor at a stable, authenticated-free endpoint and alert on both availability and latency. Do not point synthetics at the Events API — you would be consuming your own rate-limit budget to test a service you are already polling.

**Sources:** [Box status page \(Box\)](#). **Derived:** § 14.1's endpoint list and component names were read directly from the live Statuspage API on 08/25/2026.

## 15. Dashboards, Alerting, and SLOs

### 15.1 Dashboards worth building

| Dashboard              | Audience           | Content                                                                                               |
|------------------------|--------------------|-------------------------------------------------------------------------------------------------------|
| Box security posture   | Security           | Shield alerts by category and risk score; external shares; failed logins; access from new geographies |
| Box usage              | IT / business      | Active users, uploads and downloads by volume, most-active folders, storage trend                     |
| Box integration health | Observability team | Collector freshness (§ 13), duplicate rate, daily volume vs. budget, ingest failures                  |
| Your apps ↔ Box        | Application teams  | Client-span latency and error rate against <code>box.com</code> (§ 3.1)                               |

The third one is the one teams forget, and it is the one that protects the other three. A security dashboard fed by a dead collector shows an encouraging absence of alerts.

### 15.2 What deserves an alert

| Signal                                                 | Alert?                                 | Notes                                                 |
|--------------------------------------------------------|----------------------------------------|-------------------------------------------------------|
| <code>SHIELD_ALERT</code> above a risk-score threshold | Yes                                    | Page security. Route per FAQ-21                       |
| Collector stale or absent                              | Yes                                    | § 13 — highest-consequence operational alert here     |
| Failed-login spike                                     | Yes                                    | Credential-stuffing signal                            |
| External share of a regulated folder                   | Yes, if you can identify those folders | Otherwise noise                                       |
| Individual downloads                                   | No                                     | Volume without signal. Use Shield's anomaly detection |
| Box status page degradation                            | Maybe                                  | Informational unless Box is on a critical path        |

## 15.3 SLOs

Box is a vendor service — you cannot control its reliability, so an SLO on Box itself is a reporting instrument, not an engineering target. What genuinely belongs in an SLO is **your integration**:

| SLO                         | SLI                                                        | Why it is the right target                                   |
|-----------------------------|------------------------------------------------------------|--------------------------------------------------------------|
| <b>Audit completeness</b>   | % of 5-min windows with at least one successful collection | Directly measures the compliance guarantee you actually made |
| <b>Collection freshness</b> | % of time the newest event is < 15 min old                 | The detection window that keeps § 13 honest                  |

The first of those is the one an auditor will ask about. See the **SLO** series for construction.

## 16. Implementation Sequence and Checklist

### 16.1 Sequence

| Phase                    | Do                                                                           | Gate before moving on                                                   |
|--------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>1. Free signal</b>    | Run the § 3.1 span query                                                     | Know whether you already have Box performance data                      |
| <b>2. Scope</b>          | Decide which signal classes you need and why                                 | Written answer to "which questions are we buying?"                      |
| <b>3. Box side</b>       | Create the app (CCG), authorize it, grant admin + reports permission         | A manual <code>curl</code> against <code>/events</code> returns entries |
| <b>4. Dynatrace side</b> | Credential Vault entry, <code>api.box.com</code> allowlisted, bucket created | Allowlist confirmed — silent failures start here                        |
| <b>5. Collector</b>      | Build per § 7 (or § 8), run at low frequency                                 | Events arriving in the right bucket (§ 12.3)                            |
| <b>6. Processing</b>     | OpenPipeline: parse, mask, extract metrics, discard raw                      | Metrics exist <i>and</i> raw volume has dropped                         |

| Phase         | Do                                                    | Gate before moving on                                |
|---------------|-------------------------------------------------------|------------------------------------------------------|
| 7. Operate    | Staleness alert (§ 13), duplicate check, volume check | Alerts proven by deliberately stopping the collector |
| 8. Downstream | Dashboards, Shield routing, SLOs                      | —                                                    |

**Phase 7 is proven by breaking it on purpose.** An untested staleness alert is an assumption, and the cost of that assumption being wrong is a permanent, undetectable audit gap.

## 16.2 Administrator checklist

### Box side

- [ ] Box app created with **Client Credentials Grant**
- [ ] App authorized in the Box Admin Console (a fresh app is *not* authorized by default)
- [ ] Service account is enterprise admin/co-admin with **"Run new reports and access existing reports"**
- [ ] Confirmed `GET /events?stream_type=admin_logs_streaming` returns entries with a manual call
- [ ] Shield licensing confirmed (or § 11 knowingly skipped)

### Dynatrace side

- [ ] `client_id` / `client_secret` in the **Credential Vault** — not in workflow code
- [ ] `api.box.com` **on the JS-runtime outbound allowlist** (Workflow path only)
- [ ] Collector is a **standard** workflow, not simple
- [ ] Dedicated bucket created with agreed retention
- [ ] `logs.ingest` token scope (Python path only)
- [ ] OpenPipeline: parse → mask/hash → extract metrics → **No storage assignment at stage 9**, never `Drop record` at stage 1

### Operations

- [ ] Cursor persisted, **no TTL**, written only after successful ingest
- [ ] Staleness alert live **and tested by stopping the collector**
- [ ] Duplicate-rate check scheduled
- [ ] Backfill path (§ 8) built and tested *before* it is needed

- [ ] Retention and masking signed off by whoever owns the data

## 17. Common Gotchas

| #  | Gotcha                                                  | Symptom                                                 | Fix                                                                                                     |
|----|---------------------------------------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 1  | Reaching for webhooks                                   | Receiver works; the events you need never arrive        | Webhooks are file/folder-scoped and carry no audit events. Poll <code>/events</code> (§ 2.2)            |
| 2  | Host not on the outbound allowlist                      | Workflow succeeds, ingests nothing, logs nothing useful | Add <code>api.box.com</code> to <code>builtin:dt-javascript-runtime.allowed-outbound-connections</code> |
| 3  | Missing the reports permission                          | API returns 200 with no entries                         | Grant "Run new reports and access existing reports" to the service account                              |
| 4  | TTL on the cursor                                       | Collector silently resets weeks later                   | <code>stateClient</code> TTL caps at 90 days. Omit <code>validUntilTime</code>                          |
| 5  | Cursor written before ingest                            | Events vanish on a failed ingest                        | Write the cursor only after ingest succeeds (§ 7.3)                                                     |
| 6  | Not deduplicating                                       | Inflated counts and ingest cost                         | <code>admin_logs_streaming</code> repeats events. Dedupe on <code>event_id</code> (§ 12.2)              |
| 7  | <code>http.url</code> in span queries                   | Zero rows, reads as "no Box traffic"                    | The field does not exist. Use <code>server.address / url.full</code> (§ 3.1)                            |
| 8  | Staleness check returns "OK" with no data               | Dead collector reports healthy                          | <code>null &gt; 15</code> is false. Test <code>events == 0</code> explicitly (§ 13.1)                   |
| 9  | Drop record at stage 1                                  | Raw discarded <i>and</i> metrics never built            | Use <code>No storage assignment</code> at stage 9 (§ 9.2)                                               |
| 10 | <code>box-sdk-gen</code> instead of <code>boxsdk</code> | Building on a deprecated SDK                            | <code>boxsdk</code> v10+ is current (§ 8.1)                                                             |

| #  | Gotcha                           | Symptom                                    | Fix                                                                          |
|----|----------------------------------|--------------------------------------------|------------------------------------------------------------------------------|
| 11 | Simple workflow                  | <code>automation-utils</code> returns 404  | Use a standard workflow (§ 7.1)                                              |
| 12 | Backfill sharing the live cursor | Live collector jumps backwards or forwards | Separate cursor per stream type (§ 8.3)                                      |
| 13 | Renaming fields after extraction | Metrics carry stale dimension names        | Normalize in Processing, then extract (§ 10)                                 |
| 14 | 413 on ingest                    | Batch inside 10 MB still rejected          | Payload expands during processing; ceiling is 16 MB. Halve the batch (§ 9.1) |
| 15 | <code>interval:1d</code>         | Calendar-duration warning                  | Use <code>interval:24h</code> (§ 12.4)                                       |

Gotchas 1, 7, and 8 share a shape worth naming: **each produces a confident, plausible, wrong answer rather than an error**. A webhook that delivers nothing, a span query that returns zero, and a health check that says OK all look like success. Every one of them was reproduced during the research for this entry.

## 18. Recommended Approach

**Query your own spans first.** If your services call Box, that data exists now and costs nothing (§ 3.1). It may answer the question that prompted the project.

**Then build one collector, in a Workflow.** At 1M events/day a Workflow clears the 120-second ceiling with roughly 90% headroom, keeps secrets in the Credential Vault, and inherits scheduling and failure alerting. Poll `admin_logs_streaming` every 5 minutes, dedupe on `event_id`, persist the cursor with `stateClient` and no TTL, and write that cursor only after a successful ingest.

**Write the Python backfill before you need it.** It is the only recovery path for an outage longer than two weeks, and the worst time to write it is during one.

**Decide retention before switching on.** At ~45 GB/month, extract-then-discard is not an optimization — it is the difference between a proportionate cost and an unpleasant

surprise. Keep Shield alerts and regulated-folder activity; discard the [PREVIEW](#) and [DOWNLOAD](#) bulk once a counter has captured it.

**Alert on the collector, and prove the alert works by stopping it.** Everything else in this entry protects data you have. This is the only control that protects data you have not collected yet — and Box's two-week streaming window means an unnoticed outage becomes permanent.

**What "done" looks like:** Shield alerts routed to security with a working console link; a security dashboard whose freshness you can vouch for; audit records in a bucket with retention and access someone signed; and a staleness alert that has been tested by breaking it on purpose.

---

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*